INTEGRAL DRAGON

Module 2 Review & Module 3 Preview

Spec-Driven Development & Context Control → Tooling, MCP & Agentic Orchestration

AI-Accelerated Software Development | Foundations Program

Confidential. Prepared exclusively for Ensemble Health Partners.

MODULE 2 REVIEW

Spec-Driven Development & Context Control — what we covered

SESSION 5

Spec-Driven Development

The control point for intent. Write a machine-readable, testable spec before permitting any code generation. The spec — not the model — determines output quality.

SESSION 6

Plan Mode vs. Act Mode

The control point for safe execution. Force the AI to show its reasoning and propose changes before touching a single file. The human reviews and approves the plan; only then does execution begin.

SESSION 7

CLAUDE.md

The control point for durable project rules. Version-controlled standards that inject automatically into every session — no repetition, no drift, no forgotten constraints.

SESSION 8

Model Context Protocol (MCP)

The control point for external capability extension. One open standard connects your agents to every enterprise database, API, and tool — with auditable, governed access.

Module 2 teaching promise: Reliable agentic engineering depends on four things — a **clear spec**, a **reviewed plan**, **durable project rules**, and **governed tool access**.

A vague prompt produces rework — the spec is the only mechanism that makes AI output converge on correct

Spec-Driven Development

Write machine-readable, testable specifications *before* permitting code generation. When the output is flawed, the specification was flawed — not the model. The spec is the single source of truth that makes AI output auditable.

Binary Acceptance Criteria

Every spec item must be testable with a yes/no answer. "The endpoint should be fast" is useless. "The endpoint responds in under 200ms at p99 with a valid JSON body conforming to this schema" is a spec. Ambiguous specs produce ambiguous output.

Specs Break the Rework Loop

Without a spec, teams enter an infinite prompt-tweak-retry cycle. The spec is the convergence mechanism — it gives the model a precise target and gives the engineer a precise acceptance test. Both are necessary to stop iterating.

Misconception to correct

"Writing specs slows down AI development." This gets it exactly backwards. Without a spec, output never converges. The spec is not overhead — it is the only thing that makes the AI stop changing the answer.

What we built

```
Test A: "Build me a user auth endpoint"
→ Missing edge cases, inconsistent errors, no validation

Test B: Markdown PRD with 10 acceptance criteria,
            input rules, error contract, response schema
→ Complete implementation, correct edge handling,
            consistent errors, every criterion passing
```

Separating thinking from typing is the single most important safety discipline in autonomous enterprise systems

Plan Mode (Read-Only Exploration)

The AI maps the directory structure, reads relevant files, and proposes a detailed execution plan — including every file it intends to modify and exactly why. No writes. No side effects. Pure analysis before any action.

Act Mode (Supervised Execution)

Only after the engineer reviews and approves the plan does the AI gain write access. Every file modification is previewed, diffed, and confirmed. The human remains the gatekeeper at every step of the execution.

The Discipline of Separation

This is not a convenience feature. In production systems touching payroll, billing, or regulated data — the plan review IS the safety net. Making the AI's reasoning visible before it acts is the difference between a controlled system and an unchecked one.

Dangerous pattern to avoid

"I'll let it code and fix the bugs later." This is how critical files get overwritten, migrations get corrupted, and production databases receive unexpected schema changes at 11pm on a Friday.

What we practiced

```
$ claude --permission-mode plan
> "Propose a refactor of the auth middleware"
→ Claude mapped files, identified dependencies,
         listed every file it intended to touch
→ Reviewed, critiqued, requested one revision
→ Approved plan → transitioned to Act Mode
```

CLAUDE.md turns your team’s architectural standards into automated enforcement that never forgets and never needs reminding

Configuration Hierarchy

CLAUDE.md files operate at three scopes: global (~/.claude/CLAUDE.md), project-local (repo root), and directory-specific. Rules cascade and compose. Teams get fine-grained control over agent behavior at every level of the codebase.

Persistent Rule Enforcement

Instead of pasting the same system constraints into every new chat session, CLAUDE.md automatically injects your team's architectural rules, coding standards, and review requirements into every agent session. Write once, enforce forever.

Version-Controlled Governance

Because CLAUDE.md lives in the repository, it's versioned with git, reviewed in pull requests, and shared across the entire team. Your AI governance rules receive the same engineering rigor as the production code they govern.

Misconception to correct

"I have to paste my rules into every new conversation." No. CLAUDE.md eliminates this permanently. Standards persist automatically across every session without any repetition from the engineer.

What we configured

```
$ cat CLAUDE.md
# Never modify /migrations without explicit approval
# Always use single quotes and trailing commas
# Run full test suite before any commit

→ Claude autonomously follows every rule in every
        response — zero reminders needed
```

MCP connects your agents to every enterprise system through one auditable protocol — eliminating the integration tax permanently

The Model Context Protocol

An open standard that securely connects AI agents to enterprise databases, internal tools, and proprietary APIs — without custom brittle integration code. Build the connector once; every agent in the organization speaks the same language.

From Autocomplete to Enterprise Agent

Without MCP, an AI assistant is limited to what's in the prompt. With MCP, it queries live databases, reads issue trackers, accesses documentation systems, and takes governed actions — all through a standardized, auditable protocol.

Solving the Integration Tax

Before MCP, every AI tool needed custom API wrappers for every enterprise integration. MCP replaces vendor-locked plugin ecosystems with one open standard — the same way USB-C replaced a drawer full of proprietary chargers.

Misconception to correct

"Teams must write custom API wrappers for every AI tool integration." MCP eliminates this entirely. One protocol, one integration pattern, every enterprise system. The connector ecosystem is growing — most systems already have one.

What we connected

```
$ claude mcp add github
> "Show open bugs labeled 'critical' in our repo"
→ Claude queried GitHub live — no custom code

$ claude mcp add postgres
> "How many active users signed up this week?"
→ Claude queried the database directly by prompt
```

COMING NEXT — MODULE 3

Tooling, MCP, and Agentic Orchestration

Sessions 9–12: Skills, Subagents, Agent Teams, and Securing the Expanded Surface

What should stay probabilistic, and what must become deterministic?

Skills vs. Hooks — The Key Distinction

Skills are flexible, reusable workflows that augment probabilistic AI reasoning. Hooks are rigid enforcement gates that execute deterministically — regardless of what the model decides. Skills enable capability; hooks enforce non-negotiables.

Deterministic Enforcement Gates

Some behaviors cannot be negotiable: security scans, test runners, lint checks, audit log writes. Hooks enforce these unconditionally, running before and after every model action. The model doesn't get a vote on whether the hook fires.

Composable Capability Units

Skills package sophisticated agent workflows into callable, auditable units. Instead of re-prompting from scratch for recurring tasks, skills create reusable patterns — shareable across teams and composable into larger orchestrations.

The decision rule

Negotiable?
Implement as a **skill**. Flexible, reusable, probabilistic.

Non-negotiable?
Implement as a **hook**. Rigid, deterministic, unconditional.

What you'll build

```
Hook: Blocks any write to /migrations without approval
Hook: Runs test suite after every code change
Skill: /spec — full spec-driven dev workflow as one command
```

How do we keep complexity from collapsing the context window?

Atomic Task Slicing

Large problems must be decomposed into isolated, independently executable units before delegation. Ambiguous handoffs produce hallucinated work. A well-sliced atomic task has a single clear input, a single verifiable output, and no hidden dependencies.

Isolated Subagent Execution

Each subagent receives only the context required for its specific unit of work — nothing more. This prevents context rot, keeps reasoning sharp, and makes each subagent's output independently auditable before it flows into the larger result.

The Delegation Contract

Every delegation must define: what the subagent receives, what it produces, what success looks like, and what the orchestrator validates before accepting the output. Without a contract, the orchestrator has no basis for rejection.

Misconception to correct

"I can just tell the agent to do the whole feature." Complex tasks that overflow the context window silently degrade. The model won't say it's confused — it will produce plausible-looking output that misses the actual requirements.

What you'll practice

1. Decompose a billing feature into 4 atomic tasks
2. Delegate each task with a scoped context bundle
3. Validate each subagent output individually
4. Compose the verified outputs into the final result

What changes when multiple workers operate at once?

Parallel Execution Model

Multiple agents working concurrently on decomposed task units. When designed correctly, this dramatically compresses time-to-completion for complex work. The constraint is not speed — it is boundary design. Bad boundaries make concurrency dangerous.

Shared-State Risks

Concurrent agents modifying the same files, databases, or APIs can create race conditions, merge conflicts, and corrupted state. Boundary design is not optional — it is what determines whether parallel execution accelerates work or destroys it.

The Orchestrator Role

The parent agent coordinates the team — it does not write code. Its job is to decompose work cleanly, delegate with clear contracts, monitor for failures, and validate output before composition. It governs; it does not execute.

Misconception to correct

"More agents always means faster results." Without clean boundaries and state isolation, more agents means more collision surface. Concurrency amplifies design quality — good design gets faster, bad design gets worse.

What you'll build

```
Orchestrator: Decomposes and delegates 3 parallel workers
Boundaries: Explicit rules preventing shared-state collision
Validation: Each worker output checked before composition
Result: Composed, verified final artifact from 3 concurrent
streams
```

What new attack and governance surface did we just create?

Prompt Injection

Malicious instructions embedded in tool outputs, external data sources, or user input that redirect agent behavior without the engineer's knowledge. The agentic attack surface is every data source the agent reads — not just direct user input.

Excessive Agency & Privilege

Agents that can take irreversible actions — delete, deploy, publish, send — without a human review gate at the point of action. Scope every tool to the minimum access required. Irreversibility demands a human in the loop.

Insecure Data Handling

Agents that pass sensitive data through untrusted channels, log credentials in conversation history, or expose internal system context via overly permissive MCP connections. Every MCP connection is a new data flow that requires a security audit.

Delivery frame for this session

"Module 3 taught you to expand capability. Session 12 teaches you to secure the surface you just created. Every new tool connection, every subagent boundary, every hook — is also a new attack vector."

What you'll audit

- ✓ MCP config for over-provisioned access scopes
- ✓ Hooks for injection vulnerability in input handling
- ✓ Subagent contracts for unintended privilege escalation
- ✓ Orchestrator for excessive agency in irreversible actions